Problem 1: Studying the anatomy of a semaphore

1.1 Code of P and V

I use the convention from the reminders: when the counter is negative, its absolute value is the number of tasks waiting in the queue. P first takes one access and blocks the caller if none was left; V gives the access back and, if a task was waiting, unblocks it.

```
1  void P(SEM *sem)
2  {
3      TASK *t;
4      sem->count = sem->count - 1;
5      if (sem->count < 0) {          /* no access left: block the caller */
6          t = current();
7          t->state = BLOCKED;
8          insert(*t, &sem->queue);
9          sched();                   /* elect another task */
10     }
11 }
12
13 void V(SEM *sem)
14 {
15     TASK *t;
16     sem->count = sem->count + 1;
17     if (sem->count <= 0) {         /* at least one task was waiting */
18         t = extract(&sem->queue);
19         t->state = READY;
20     }
21 }
```

I assume `insert` records the task itself, not a copy, so `extract` later returns the same task. In V, the test `count` ≤ 0 after the increment means `count` < 0 before, which is exactly the case where the queue is not empty. The unblocked task does not test the counter again: when it is elected, it returns from `sched()` inside P and already owns the access. V does not call `sched()`; the unblocked task becomes READY and is elected later by the scheduler. With a priority scheduler, V could also call `sched()` so that a more urgent task runs at once.

1.2 Why P and V must be indivisible

P and V read and modify shared data, the counter and the queue. If two calls are interleaved, the semaphore can become inconsistent. Two examples:

- **Two P at the same time.** With `count` $= 1$, A reads the counter and is interrupted before writing 0. B executes all of P: `count` $= 0$ and B enters its critical section. A resumes, writes $1 - 1 = 0$, finds `count` ≥ 0, and also enters. Both tasks are in the critical section.

- **Lost wake-up.** With `count` $= 0$, A executes `count - 1`, so `count` $= -1$, but is interrupted before `insert`. B, in the critical section, calls V: `count` $= 0 \leq 0$, so it extracts from the queue, which is still empty. A then inserts itself and calls `sched()`: it stays blocked although the resource is free, and no V will wake it.

P and V are system calls, so their code runs in kernel mode.

- On a uniprocessor, the kernel masks interrupts at the start of P and V and unmasks them at the end. No other task can then be elected in the middle of the code, except at the explicit `sched()` in P, where the new task restores its own interrupt state.
- On a multiprocessor, masking interrupts only affects the local processor, and another processor can run P or V on the same semaphore. Each semaphore is then also protected by a spinlock built on an atomic instruction (test-and-set, compare-and-swap, exchange). This is busy waiting, but it is acceptable because P and V are only a few instructions long. A task that blocks in P must release the spinlock when it is switched out.

1.3 Masking interrupts in user mode

- Masking interrupts is a privileged instruction. If user code could do it, a process that never unmasks them (a bug, an infinite loop in the critical section, or malicious code) would also block the clock interrupt. It could no longer be preempted, and the OS would lose control of the CPU.
- Device interrupts are delayed for the whole critical section. A user critical section can be long, unlike the few instructions of P and V.
- It is too coarse: it stops every other process, including the ones that never use the resource. A semaphore only delays the processes that need the same resource.
- It does not work on a multiprocessor: masking interrupts on one processor does not prevent a process on another processor from entering its critical section.

1.4 Interrupting a process in a critical section

Yes, it can be interrupted. A critical section protected by a semaphore runs in user mode with interrupts enabled. Indivisibility is only with respect to the resource: no other critical section on the same resource may interleave with it. The process can be interrupted by the clock at the end of its quantum, by an I/O completion, or because a task with higher priority becomes ready.

When this happens, the kernel saves the context of the process and puts it back in the READY state. It still holds its access, and the counter of the semaphore does not change. The scheduler may then elect another process:

- If that process does not use the resource, it runs normally.
- If it tries to enter a critical section on the same resource, its P makes the counter negative. It is blocked in the semaphore queue, and another task is elected. It never enters the critical section.

Later, the interrupted process is elected again, continues its critical section where it stopped, and calls V, which unblocks one waiting process. Mutual exclusion is therefore preserved, but the waiting processes are delayed for as long as the holder is preempted. This is one reason to keep critical sections short.

Problem 2: First steps with semaphores

I label the instructions as in the problem sheet. In each program the letters run from top to bottom in Process A, then B, then C. I assume each labeled instruction is executed indivisibly and the semaphore queues are FIFO. In the tables, r means ready, b means blocked, and end means the process has finished; – marks an empty queue.

2.1 Hand execution with Mutex

Process A is (a) `P(Mutex)`, (b) `x = x + 1`, (c) `V(Mutex)`; B is (d), (e) `x = x * 2`, (f); C is (g), (h) `x = x - 4`, (i). The counter of Mutex starts at 1, and x_0 is the initial value of x.

Sequence a d b g c e f h i.

Step	Instruction	Counter	Queue	A	B	C	x
	initial	1	–	r	r	r	x_0
(a)	A: `P(Mutex)`	0	–	r	r	r	x_0
(d)	B: `P(Mutex)`	-1	B	r	b	r	x_0
(b)	A: `x = x + 1`	-1	B	r	b	r	$x_0 + 1$
(g)	C: `P(Mutex)`	-2	B, C	r	b	b	$x_0 + 1$
(c)	A: `V(Mutex)`	-1	C	end	r	b	$x_0 + 1$
(e)	B: `x = x * 2`	-1	C	end	r	b	$2x_0 + 2$
(f)	B: `V(Mutex)`	0	–	end	end	r	$2x_0 + 2$
(h)	C: `x = x - 4`	0	–	end	end	r	$2x_0 - 2$
(i)	C: `V(Mutex)`	1	–	end	end	end	$2x_0 - 2$

This sequence is valid. Each instruction is executed by a process that is not blocked, and the critical sections do not overlap: A is inside on $[a, c]$, B on $[c, f]$, and C on $[f, i]$. At (c), V unblocks B because B is at the head of the FIFO queue; with a policy that unblocked C first, (e) could not come next. The final value is $x = 2(x_0+1)-4 = 2x_0-2$.

Sequence a d e b c f g h i. After (a) the counter is 0, and after (d) it is -1 with B blocked in the queue. Instruction (e) belongs to B, which is blocked in `P(Mutex)` and can only continue after a `V(Mutex)`. The first V is (c), so (e) cannot come before (c). This sequence is **not valid**: it would put A and B in their critical sections at the same time.

2.2 Valid sequences of (b), (d), (g)

Now Process A is (a) `P(S1)`, (b) `a = a + 7`, (c) `V(S2)`; B is (d) `a = a - 5`; C is (e) `V(S1)`, (f) `P(S2)`, (g) `a = a * 3`. Initially $a = 6$, $S1 = 1$, and $S2 = 0$.

- S1 starts at 1, and the only `P(S1)` is (a), executed once. So (a) never blocks.
- S2 starts at 0, so (f) blocks C until A executes (c). Thus (g) comes after (c), hence after (b).
- (d) is not synchronized.

The only constraint is that (b) comes before (g). Out of the $3! = 6$ orders, three are valid:

Order	Computation	Final value of a
b d g	$(6 + 7 - 5) \times 3$	24
d b g	$(6 - 5 + 7) \times 3$	24
b g d	$(6 + 7) \times 3 - 5$	34

The orders g b d, g d b and d g b are not valid, because (g) would run before (c). So the final value of a is either 24 or 34.

2.3 Semaphore that can be deleted

S1 can be deleted. Its counter starts at 1, the only `P(S1)` is (a), executed once, and `V(S1)` only increases the counter. So `P(S1)` never blocks, and S1 never changes which sequences are possible. Removing (a), (e) and the creation of S1 leaves the same valid sequences and values as in 2.2.

S2 cannot be deleted, because it is what forces (g) after (b). Without it, the orders g b d, g d b and d g b become possible, and they give $6\times3+7-5=20$, $6\times3-5+7=20$, and $(6-5)\times3+7=10$.

2.4 Always obtaining $a = 34$

From 2.2, $a = 34$ only for the order b g d, so (d) must also wait for (g). I add `S3 = CS(0)`: C executes `V(S3)` after (g), and B executes `P(S3)` before (d).

```
int a = 6; S1 = CS(1); S2 = CS(0); S3 = CS(0);
```

Process A	Process B	Process C
`P(S1);`	`P(S3);`	`V(S1);`
`a = a + 7;`	`a = a - 5;`	`P(S2);`
`V(S2);`		`a = a * 3;`
		`V(S3);`

S2 forces (b) before (g), and S3 forces (g) before (d). The only possible order is b g d, so $a = (6 + 7) \times 3 - 5 = 34$. There is no deadlock: A never blocks, C only waits for A, and B only waits for C.

2.5 Always obtaining $a = 24$

From 2.2, $a = 24$ when (d) comes before (g), so (g) must also wait for (d). I add `S3 = CS(0)`: B executes `V(S3)` after (d), and C executes `P(S3)` before (g).

```
int a = 6; S1 = CS(1); S2 = CS(0); S3 = CS(0);
```

Process A	Process B	Process C
`P(S1);`	`a = a - 5;`	`V(S1);`
`a = a + 7;`	`V(S3);`	`P(S2);`
`V(S2);`		`P(S3);`
		`a = a * 3;`

Now (g) comes after both (b) and (d). The order of (b) and (d) does not matter, since $6 + 7 - 5 = 6 - 5 + 7 = 8$, so $a = 8 \times 3 = 24$. A and B never block, so C's two P operations are always eventually satisfied.

2.6 A sequence leading to a deadlock

In the third program, Process A is (a) `P(S1)`, (b) `P(S2)`, (c) `a = a + 7`, (d) `V(S1)`, (e) `V(S2)`; B is (f) `P(S2)`, (g) `a = a - 5`, (h) `V(S2)`, (i) `V(S1)`; C is (j) `P(S2)`, (k) `P(S1)`, (l) `a = a * 3`, (m) `V(S2)`. Both semaphores start at 1.

A takes S1 then S2, while C takes them in the opposite order. The sequence **a j b k f** leads to a deadlock:

Step	Instruction	S1	Queue S1	S2	Queue S2	A	B	C
	initial	1	–	1	–	r	r	r
(a)	A: `P(S1)`	0	–	1	–	r	r	r
(j)	C: `P(S2)`	0	–	0	–	r	r	r
(b)	A: `P(S2)`	0	–	−1	A	b	r	r
(k)	C: `P(S1)`	−1	C	−1	A	b	r	b
(f)	B: `P(S2)`	−1	C	−2	A, B	b	b	b

All three processes are blocked. A waits for S2, which C holds, and C waits for S1, which A holds: this is a circular wait. B also waits on S2. Only (d), (e), (i) or (m) could unblock someone, and they all belong to blocked processes, so no process can progress. B could have saved the situation with its `V(S1)` in (i), but it blocks at (f) before reaching it.

Problem 3: Synchronization and deadlock

I assume `workworkwork()` always terminates, and the semaphore queues are FIFO.

3.1 Maximum number of processes using a resource

The answer is Y. A process uses a resource between `P(R)` in the prologue and `V(R)` in the epilogue. Since R starts at Y, at most Y processes can have passed `P(R)` without executing `V(R)`. This bound is reached: each process releases its server right after getting its resource, so Y processes can pass the prologue one after another and work at the same time ($N > Y$). The servers do not limit this number, since a server is only held during the prologue and the epilogue, not during `workworkwork()`.

3.2 Deadlock for $N = 2$ and $X = Y = 1$

Here `S = CS(1)` and `R = CS(1)`. Consider the following execution:

Step	Process	Operation	S	R	U1	U2
		initial	1	1	r	r
1	U1	`P(S)`	0	1	r	r
2	U1	`P(R)`	0	0	r	r
3	U1	`V(S)`, starts working	1	0	r	r
4	U2	`P(S)`	0	0	r	r
5	U2	`P(R)`	0	-1	r	b
6	U1	ends working, `P(S)`	-1	-1	b	b

U1 holds the resource and waits for the server, while U2 holds the server and waits for the resource. The `V(R)` that would unblock U2 comes after `P(S)` in U1's epilogue, and the `V(S)` that would unblock U1 comes after `P(R)` in U2's prologue. Neither can progress, so this is a deadlock.

3.3 Minimum value of N for a deadlock

The minimum is $N = X + Y$.

$N = X + Y$ **can deadlock.** This generalizes 3.2:

(1) Y processes pass the prologue one after another and start working: $R = 0$ and $S = X$.

(2) X other processes execute `P(S)`, then `P(R)`: $S = 0$, and they all block on R ($R = -X$), each holding a server.

(3) The Y working processes finish and execute `P(S)` in the epilogue: they all block ($S = -Y$).

All $X + Y$ processes are blocked.

$N < X + Y$ **cannot deadlock.** Suppose some process is blocked forever.

- If it is blocked on `P(S)`, the counter of S stays ≤ 0, so all X servers are held forever. A server holder in the epilogue only executes `V(R)` and `V(S)`, which never block, so it would release its server. So all X server holders are in the prologue, blocked forever on `P(R)`.
- If it is blocked on `P(R)`, all Y resources are held forever. Their holders are past `P(R)` and before `V(R)`. Since `workworkwork()` terminates, they must be blocked forever on `P(S)` in the epilogue.

Either case leads to the other, so a deadlock needs X processes blocked on `P(R)` while holding the servers, and Y processes holding the resources. These are different processes, because the first group does not have a resource yet. A deadlock therefore needs at least $X + Y$ processes.

3.4 Preventing the deadlock with a semaphore L

I take `L = CS(X + Y - 1)` and place `P(L)` at the start of the prologue and `V(L)` at the end of the epilogue:

```
1 S = CS(X); R = CS(Y); L = CS(X + Y - 1);
2 Begin loop
3   P(L); P(S); P(R); V(S);     /* Prologue */
4   workworkwork();             /* Resource usage */
5   P(S); V(R); V(S); V(L);     /* Epilogue */
6 End loop
```

L counts the processes between the start of the prologue and the end of the epilogue, which are exactly the processes that can hold or wait for a server or a resource. At most $X+Y-1$ of them are past `P(L)`, so by 3.3 they cannot deadlock among themselves. The processes waiting on `P(L)` hold nothing, and they are unblocked by `V(L)` as the others finish their epilogue.

The positions matter:

- `P(L)` must come before `P(S)`. Otherwise a process could hold a server while waiting on L, while a process in its epilogue, which holds an access to L, waits for that server. That would create a new deadlock.
- `V(L)` must come after `V(R)`, since the processes in `workworkwork()` hold resources and are part of the deadlock in 3.3.

Any initial value $1 \leq L \leq X+Y-1$ prevents the deadlock. For example, with $L = Y$ at most Y processes compete for resources, so `P(R)` never blocks. I choose $X + Y - 1$ because it is the largest value and restricts concurrency the least. Since $X \geq 1$, we have $Y \leq X + Y - 1$, so Y processes can still use the resources at the same time as in 3.1.